

AI WEATHERWISE

Intelligent Weather Forecasts and AI-Powered Insights

1. Introduction

AI WeatherWise is a robust RESTful backend platform designed to deliver real-time weather metrics together with intelligent summaries and personalized recommendations. The system is built using Node.js and Express.js, with MongoDB and Mongoose ODM used for data persistence. Google Gemini AI is integrated to convert live environmental conditions into natural-language weather insights and activity recommendations.

The project addresses the difficulty of interpreting raw weather measurements and managing weather information across different cities. AI WeatherWise provides a centralized service where users can manage favorite locations, retrieve current weather information, and request AI-generated insights.

2. Problem Statement

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural-language weather summaries instead of only raw weather metrics.
- Need for a secure system to store and synchronize user preferences across devices.

3. Proposed Solution

The proposed system combines weather-data retrieval, centralized user and location management, authentication, and AI orchestration. Registered users can save favorite cities, retrieve current weather metrics, and request Gemini-powered summaries and recommendations. The backend includes JWT authentication, bcrypt password hashing, request sanitization, centralized error handling, and a fallback mode for external API-key availability.

4. Project Outcome

By centralizing weather data and applying AI-based interpretation, the system reduces the effort required to understand raw environmental metrics. Mongoose schemas support data integrity, while Gemini integration provides context-aware weather advice. The architecture is organized for maintainability and scalability through separate routing, controller, middleware, model, and utility components.

5. Software and Hardware Requirements

The project specifies a cross-platform environment consisting of Windows 10/11, macOS, or Linux. Node.js v16 or above provides the runtime environment, while npm v8 or above manages dependencies. The project uses the Gemini AI SDK, MongoDB, Postman, and Visual Studio Code or a similar code editor.

- Operating System: Windows 10/11, macOS, or Linux.
- Runtime: Node.js v16 or above; Package Manager: npm v8 or above.
- Database: MongoDB; AI Integration: Gemini AI SDK.
- API Testing: Postman; Development Tool: Visual Studio Code or similar IDE.
- Processor: Intel Core i5 8th Gen+ / AMD Ryzen 5 or equivalent.
- Memory: 8 GB minimum; 16 GB recommended.
- Storage: 1 GB available workspace.

6. System Architecture

AI WeatherWise follows the MVC pattern to separate business logic from data management. The Controller layer coordinates weather retrieval and AI insight generation. The Model layer manages User and Location entities through Mongoose schemas. In this headless backend service, the interface layer is represented by API routing, which maps network requests to controller handlers.

Major components include the Express Server Gateway, authentication and role middleware, granular service modules, the Mongoose database interface, and AI service integration. Express handles HTTP methods such as GET, POST, PUT, and DELETE, JSON parsing, and CORS. Authentication middleware validates JWT tokens and checks authorization requirements.

7. Database Design

The data architecture contains User and Location entities. A User has an ObjectId, name, unique email, hashed password, and role. A Location contains its own ObjectId, a reference to the User collection, city, country, and creation timestamp. The relationship is one-to-many: one registered user can save multiple favorite locations.

8. Project Setup and Configuration

The project is initialized as a Node.js application and developed in Visual Studio Code. The documented setup initializes the npm project and installs Express, Mongoose, bcryptjs, jsonwebtoken, cors, dotenv, multer, and the Google AI package. Nodemon is installed as a development dependency.

Environment configuration is maintained in a .env file. The documented variables include PORT, MONGO_URI, JWT_SECRET, and GEMINI_API_KEY. This keeps connection details and secrets separate from application source code.

9. Backend Development

The backend is organized as a modular REST API using Node.js, Express.js, and MongoDB. Routing, controllers, middleware, models, and utility modules are separated to improve maintainability and scalability. The application exposes secure APIs for authentication, weather operations, favorite-location management, AI-powered insights, and administrative operations.

The index.js file acts as the application entry point. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after the database connection is successful.

10. Authentication and Security

JWT authentication is used to validate user sessions and protect guarded resources, while bcrypt password hashing is used for credential protection. Role-based route guarding checks whether a user has the required authorization level. The project also documents request sanitization as a safeguard against XSS vectors and NoSQL parameter-manipulation injections.

The user flow begins with registration or login. After authentication, the user can save a favorite city, retrieve current weather metrics, and request AI-driven insights. The system validates the JWT and returns the requested information.

11. AI Integration

Google Gemini is used to generate natural-language weather summaries and personalized recommendations from live temperature and weather conditions. This AI layer changes the interaction from simply displaying measurements to providing understandable, context-aware information that can support activity and clothing decisions.

12. Database Configuration and API Testing

MongoDB is used as the document database and Mongoose provides the application-level database interface. The database connection is established before backend actions are processed. Mongoose schemas define the User and Location structures and support validation and persistence. The project document also describes API testing through Postman or Thunder Client.

The documented execution sequence is: install dependencies using `npm install`, ensure MongoDB is running, configure the required environment variables, and start the development server using `npm start`. After successful execution, the backend connects to MongoDB and listens for incoming API requests.

13. API Modules

- Register API – supports user registration.
- Login API – authenticates users and supports protected access.
- Weather API – retrieves current weather information.
- Recommendation API – provides AI-driven weather recommendations.
- Favorite Location operations – allow registered users to manage saved locations.

14. User and Administrator Roles

The Admin role is responsible for user account management, API health and system-log oversight, and database-integrity monitoring. Registered users manage their favorite locations, request real-time weather information, interact with Gemini for personalized summaries and activity recommendations, and manage their authentication credentials.

15. Conclusion

AI WeatherWise demonstrates how a modern backend can combine REST APIs, NoSQL persistence, secure authentication, external weather services, and generative AI. Its MVC-based organization separates data models, controller logic, and routing responsibilities, making the application easier to maintain and extend. The system focuses on turning live weather information into understandable and personalized insights.

The project provides a structured development and testing workflow. Node.js and Express.js handle server-side operations, MongoDB and Mongoose manage persistent data, JWT and bcrypt support authentication security, and Gemini provides AI-powered interpretation. Postman or Thunder Client can be used to validate REST endpoints.

16. References / Project Resources

- AI WeatherWise project documentation provided for this report.
- API testing video and backend demo resources referenced in the project document.